CODE :-

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff, exp;
    struct Node *next;
};

struct Node *createNode(int c, int e) {
    struct Node *n = malloc(sizeof(struct Node));
    if (n == NULL) exit(1);
    n->coeff = c;
    n->exp = e;
    n->next = n;
    return n;
}

void insertTerm(struct Node **head, int c, int e) {
    if (c == 0) return;

    if (*head == NULL) {
        *head = createNode(c, e);
        return;
    }

    struct Node *cur = *head, *prev = NULL;

    do {
```

```
    if (cur->exp == e) {
        cur->coeff += c;
        if (cur->coeff == 0) {
            if (cur->next == cur) {
                free(cur);
                *head = NULL;
            } else {
                if (cur == *head) {
                    struct Node *last = *head;
                    while (last->next != *head)
                        last = last->next;
                    *head = cur->next;
                    last->next = *head;
                } else {
                    prev->next = cur->next;
                }
                free(cur);
            }
        }
        return;
    }

    if (cur->exp < e) break;
    prev = cur;
    cur = cur->next;
} while (cur != *head);

struct Node *n = createNode(c, e);

if (cur == *head &&
    (prev == NULL || prev->exp < e)) {
```

```
        struct Node *last = *head;
        while (last->next != *head)
            last = last->next;
        n->next = *head;
        last->next = n;
        *head = n;
    } else {
        n->next = cur;
        prev->next = n;
    }
}

struct Node *createPolynomial() {
    struct Node *head = NULL;
    int n, c, e;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter coefficient and exponent: ");
        scanf("%d %d", &c, &e);

        if (e < 0) {
            printf("Exponent must be non-negative.\n");
            i--;
            continue;
        }

        insertTerm(&head, c, e);
    }
```

```
    return head;
}

void displayPolynomial(struct Node *head) {
    if (head == NULL) {
        printf("0");
        return;
    }

    struct Node *cur = head;
    int first = 1;

    do {
        int c = cur->coeff;
        int e = cur->exp;

        if (c != 0) {
            if (!first)
                printf(c > 0 ? " + " : " - ");
            else if (c < 0)
                printf("-");

            int a = abs(c);

            if (e == 0)
                printf("%d", a);
            else {
                if (a != 1) printf("%d", a);
                printf("x");
                if (e != 1) printf("^%d", e);
            }
```

```
            first = 0;
        }
        cur = cur->next;
    } while (cur != head);

    if (first) printf("0");
}

struct Node *addPolynomial(struct Node *p,
                           struct Node *q) {
    struct Node *result = NULL;

    if (p != NULL) {
        struct Node *cur = p;
        do {
            insertTerm(&result, cur->coeff, cur->exp);
            cur = cur->next;
        } while (cur != p);
    }

    if (q != NULL) {
        struct Node *cur = q;
        do {
            insertTerm(&result, cur->coeff, cur->exp);
            cur = cur->next;
        } while (cur != q);
    }

    return result;
}
```

```
void freePolynomial(struct Node *head) {
    if (head == NULL) return;

    struct Node *cur = head->next;
    while (cur != head) {
        struct Node *temp = cur;
        cur = cur->next;
        free(temp);
    }
    free(head);
}

int main() {
    struct Node *p1, *p2, *sum;

    printf("Create first polynomial\n");
    p1 = createPolynomial();

    printf("\nCreate second polynomial\n");
    p2 = createPolynomial();

    printf("\nFirst polynomial: ");
    displayPolynomial(p1);

    printf("\nSecond polynomial: ");
    displayPolynomial(p2);

    sum = addPolynomial(p1, p2);

    printf("\nSum: ");
    displayPolynomial(sum);
```

```
    printf("\n");

    freePolynomial(p1);
    freePolynomial(p2);
    freePolynomial(sum);

    return 0;
}
```

Output :-

```
Create first polynomial
Enter number of terms: 3
Enter coefficient and exponent: 3 2
Enter coefficient and exponent: 2 1
Enter coefficient and exponent: 1 0

Create second polynomial
Enter number of terms: 3
Enter coefficient and exponent: 4 2
Enter coefficient and exponent: -2 1
Enter coefficient and exponent: 5 0

First polynomial: 3x^2 + 2x + 1
Second polynomial: 4x^2 - 2x + 5
Sum: 7x^2 + 6
```

WRITEUP :-

Assignment no :- 6

Aim :-
Write a program to implement polynomial operations using a circular linked list. The program should support the following operations:
1) Create a polynomial by inserting terms.
2) Display the polynomial in a readable mathematical form.
3) Add two polynomials and display the resultant polynomial.

Objectives :-
1. To implement a polynomial representation using a Circular linked list by dynamically creating & storing polynomial terms.
2. To perform fundamental polynomial operations such as creating, displaying and adding two polynomials using Circular linked list techniques.
3. To understand the advantages of Circular linked list in managing dynamic data structures and efficiently handling polynomial arithmetic by combining like terms.

Implementation :-

→ Platform :-
- 64-bit Open source linux or its derivatives
- Open source C programming tool like gcc/Eclipse.

→ Test Cond":
1) Implement a polynomial representation using a circular linked list.
- Create a polynomial with multiple terms entered in sequence.
- Create a polynomial with a single term.
- Create an empty polynomial with 0 terms.
- Verify that the last node points back to the first node.

2) Perform polynomial operations:
- Display a polynomial with multiple terms.
- Add two polynomials having common exponents.
- Add two polynomials having no common exponents.
- Add polynomials with -ve coefficients.

3) Understand efficient handling of polynomial arithmetic using Circular Linked List.
- Perform repeated traversal of the polynomials.
- Verify memory allocation for dynamic node creation.
- Display the resultant polynomial after add°.

Pseudo code :-
1) create_polynomial
S-1 Start
S-2 Create an empty Circular Linked List.
S-3 Input the no of terms n.
S-4 Repeat for each term from 1 to n:
(a) Input coefficient & Exponent.
(b) If coefficient is not zero :-
(i) Create a new node
(ii) Store coefficient and exponent.
(iii) Insert the node in descending order of exponents.
(iv) If the exponents already exists, combine the coefficients.
S-5 Link the last node back to the first node.
S-6 Return the polynomial.
S-7 Stop

2) display_polynomial
S-1 Start
S-2 If Head is NULL:
(a) Print "zero polynomial"
(b) Return
S-3 Set TEMP = HEAD.
S-4 Repeat
(a) Display the coefficient & exponent in mathematical form
(b) Move TEMP to TEMP->NEXT.
S-5 Continue until TEMP = HEAD.
S-6 STOP

3) Add_polynomial
S-1 Start
S-2 Create an empty resultant polynomial R.
S-3 Set P = P1 and Q = P2
S-4 Compare the exponents of P & Q:
(a) If exponents are equal:
(i) Add their coefficients
(ii) If the sum is not zero, insert the term into R.
(iii) Move P & Q to their next nodes.
(b) If P's exponents is greater:
(i) Copy P's term into R
(ii) Move P to its next node.

S(a) Otherwise:
(i) Copy Q's term into R.
(ii) Move Q to its next node
S-5 Continue until all terms of both polynomials are processed
S-6 Copy any remaining terms into R.
S-7 Return R.
S-8 Stop.

Time Complexity :-

Operation	Time Complexity
Create polynomial	$O(n^2)$
Display polynomial	$O(n)$
Add polynomial	$O(m+n)$

Conclusion :-
The program successfully implements polynomial creation, display and add° using a circular linked list, demonstrating efficient dynamic memory management and polynomial manipulation.

Faqs :-

Q1. Why is a circular linked list used to represent polynomials instead of a linear linked list?
→ It allows continuous traversal because the last node points back to the first node. It supports dynamic memory allocation and makes polynomial operations convenient.

Q2. What information is stored in each node of the Circular linked list?
→ Each node stores three fields :- the coefficient, its exponent and a pointer to the next node.

Q3. What happens if one of the polynomial is empty during the add° operation :-
→ If one polynomial is empty, the resultant polynomial is simply the other polynomial. If both are empty, the result is an empty polynomial.